

# Klasifikacija opisa filma prema njihovom žanru

## 1. Definicija problema

Problem koji treba rešiti je razvoj algoritma za automatsko određivanje žanra filma na osnovu opisa filma. Ovaj problem zahteva korišćenje tehnika obrade prirodnog jezika kako bi se analizirali tekstualni opisi i klasifikovali u odgovarajući žanr filma. Cilj je stvoriti tačan i precizan algoritam koji može automatski dodeliti žanr filmu na osnovu njegovog opisa, što bi omogućilo efikasnije kategorizovanje filmova i olakšalo korisnicima pretragu i preporuke filmova.

## 2. Motivacija

Rešenje ovog problema nam može značajno poboljšati brzinu i preciznost pretrage filmova na online platformama. Koristeći ovaj algoritam, korisnici ne moraju unapred znati tačan žanr filma koji traže. Umesto toga, mogu opisati sadržaj koji ih interesuje, a algoritam će analizirati taj tekst i dodeliti odgovarajući žanr. Na taj način, korisnicima će biti olakšano pronalaženje žanra koji odgovaraju njihovim zahtevima, bez potrebe da unose specifične ključne reči ili žanrove.

## 3. Skup podataka

Ovaj dataset sadrži informacije o filmovima, među kojima su naslov filma, njihov opis i žanr kom pripadaju.

- Broj instanci: Ovaj dataset se sastoji od 54200 instanci, što nam je dobro kako bi nam rezultati bili što bolji.
- Atributi: Atributi ovog dataseta su naslov filma, opis i žanr. Naslov filma nam ne igra veliku ulogu u ovom problemu, Glavni atributi su nam opis filma, koji ćemo koristiti za analizu i klasifikaciju žanra. Opis filma sadrži tekst koji sadrži informacije o radnji, karakterima i atmosferi filma koje su bitne za određivanje njegovog žanra. Atribut žanr predstavlja ciljno obeležje ovog problema, te na osnovu njega zaključujemo koji će biti output našeg problema.

Link do dataseta:

<https://www.kaggle.com/code/imgowthamg/movie-genre-classification/output>

## 4. Pretprocesiranje podataka

Pošto koristimo tri različita algoritma, a njihovo pretprocesiranje je drugačije u sva tri algoritma, to ćemo objasniti u delu metodologija.

## 5. Metodologija

Naš plan je da probamo više algoritama mašinskog učenja i da ih uporedimo kako se oni ponašaju, kao i da vidimo koji algoritam od njih daje najbolje rezultate. Algoritmi koje bismo upoređivali su: Naivni Bajes, K najbližih suseda i rekurentna neuronska mreža.

### 1. Naivni Bajes

Kod ovog modela ćemo koristiti bag of words za računanje verovatnoće za tekst. Za ulaz će biti korišćen tekst  $d$ , i skup klasa  $c$  koje pripada  $\{c_1, c_2, \dots, c_j\}$ , te će problem pripadati nekoj od klasa  $c$ . U našem slučaju će  $d$  biti tekst iz opisa filma, a klase  $c$  će biti žanrovi filmova (output). Ovaj problem ne zavisi od redosleda reči u tekstu, te se tu može naći problem. Formiranje bag of words se zasniva na tome da se za određeni tekst računa učestalost određenih reči u tekstu. Tu može biti problem jer se određene reči ponavljaju više puta, poput (the, a, i slicnih ucestalih reci), koje nemaju neke određeno značenje za domen problema, pa bismo taj tekst pretprocesirali, tako što bismo odabrali važne reči iz domena, pa na osnovu toga računali frekvenciju. Svaka klasa će dobiti određenu verovatnoću, tj verovatnoću da taj tekst pripada toj klasi, pa će onda taj tekst pripadati klasi koju ima najveću verovatnoću.

### 2. K najbližih suseda (KNN)

Za ovo ćemo koristiti  $n$ -grame (stringove dužine  $n$ ).  $N$ -gramski profil je lista uređenih parova ( $n$ -gram, frekvencija), gde su frekvencije izračunate za sve  $n$ -grame. Osnovna prednost korišćenja  $n$ -grama su robusnost (na primer, nisu mnogo osetljivi na greške u kucanju ili na pojavljivanje reči u različitim gramatičkim oblicima), nezavisnost od domena koji se analizira, efikasnost (dovoljan je jedan prolaz kroz tekst) i jednostavnost. Mana je eksponencijalna zavisnost broja mogućih  $n$ -grama u odnosu na dužinu  $n$ -grama.

Na osnovu tih vrednosti, možemo uočiti koji najfrekventnije  $n$ -agrame. Odvojiti sve klase kao najfrekventnijih i na osnovu toga računamo  $d$ -rastojanja i traži se najbliža rastojanja.

### 3. Rekurentna neuronska mreža - Many to One arhitektura

Prednost ovog algoritma u odnosu na prethodna dva je jer je ovom bitno redosled reči, kao i jer prethodne reči utiče na značenje. Parametri se mogu pretvoriti u one-hot embedding reprezentaciju. Za izlazni sloj moramo imati jedan neuron (jer je Many-One arhitektura). Koristi se many-one arhitektura, jer imamo više reči, a one kao izlazni neuron jer je problem klasifikacije (imaćemo jedan neuron (vektor veličine brojem različitih klasa problema)).

## 6. Način evaluacije

Za sva 3 algoritma ćemo podeliti skup na trening i testni skup. Dataset ćemo podeliti na 80% trening skup i 20% testni skup. Koristićemo različite metrike za svaki algoritam:

1. Naivni Bajes: Za ovaj algoritam koristićemo Laplace (add-1) smoothing. Tu metriku moramo da imamo, jer postoji mogućnost da se neka reč ne nalazi u tekstu, pa to moramo skalirati uz pomoć ove metrike.
2. KNN: Za ovaj problem moramo računati rastojanja između njih, pa ćemo za to koristiti Euklidsko rastojanje.

3. Rekurentna neuronska mreža: Za izlazni sloj možemo skalirati podatke na verovatnoće, uz pomoć nekih aktivacionih funkcija za višeklasnu klasifikaciju, npr softmax, , koji na osnovu vrednosti iz vektora, pretvara u verovatnoće i na osnovu toga odrediti kojoj klasi pripada dati problem.

Naime, kako Naivi Bajes i KNN su jednostavni algoritmi, za razliku od rekurentne neuronske mreže, treba proveriti tačnost algoritma, kao i odnos koliko su im tačni i precizni rezultati, kao i da li ima drastične razlike u njima.

Za to ćemo koristiti matricu konfuzije. Na osnovnu matrice ćemo naći precision i recall.

Kako bismo imali balans između precision-a i recall-a, koristićemo F meru (harmonijska sredina između te dve veličine).

Pošto je ovo problem višeklasne klasifikacije, moramo izračunati jednu F meru za ceo problem, i to možemo raditi sa:

1. Makro F mera - aritmetička sredina za svaku F meru (ovo se radi kada su svi problemi klase jednake važnosti).
2. Mikro F mera - spajanje sve u jednu matricu, jer ovde dominiraju klase koje su frekventnije.

## 7. Tehnologije

Programski jezik koji ćemo koristiti je Python zbog biblioteka koje nam nudi. Za rekurentnu neuronsku mrežu ćemo koristiti Pytorch, jer je to najbolji alat za korišćenje neuronskih mreža. Scikit-learn ćemo koristiti za algoritme Naivnog Bajesa i KNN.

## 8. Literatura

Link do dataseta i do primera kako se problem rad algoritmom Naivnog Bajesa je:

<https://www.kaggle.com/code/imgowthamg/movie-genre-classification/notebook>